NYC ROAD SAFETY INTELLIGENCE

1. **Introduction**

This document presents an initial overview of a sample of motor-vehicle crash records. The supplied table contains crash date and time, location details, injury and fatality counts, contributing factors, and vehicle-type fields. The sample shown contains five records year (2024 -2026).

Objective:

The main objective of this project is to analyze motor vehicle crash data to identify accident patterns, major contributing factors, high-risk locations, and the severity of road accidents. Using Python, Pandas, Matplotlib, and Seaborn, the project aims to transform raw crash data into meaningful insights that can help improve road safety and reduce accidents, injuries, and fatalities.

2. Dataset structure

Column group	Examples of fields	Purpose
Date and time	CRASH DATE, CRASH TIME	Identify when the crash occurred.
Geographic details	BOROUGH, ZIP CODE, LATITUDE, LONGITUDE, LOCATION	Describe where it occurred.
Road details	ON STREET NAME, CROSS STREET NAME, OFF STREET NAME	Record road or intersection information
Casualties	NUMBER OF PERSONS INJURED/KILLED; pedestrian, cyclist and motorist counts	Measure reported injuries and deaths.
Crash factors	CONTRIBUTING FACTOR VEHICLE 1 and 2	Capture reported contributing factors.
Vehicle details	VEHICLE TYPE CODE 2, 3, 4 and related vehicle fields	Describe involved vehicle types.

3.Data types to check

Field type	Suggested type / treatment
CRASH DATE	Convert to datetime after confirming the date format.
CRASH TIME	Parse as time or retain as text initially; validate values such as HH: MM.
ZIP CODE	Keep as string to preserve leading zeros and avoid treating it as a measurement.

LATITUDE / LONGITUDE	Numeric (float); check valid geographic ranges and missing values.
Injury / fatality counts	Numeric integers; inspect blanks, negative values, and implausible counts.
Borough, street, contributing factor, vehicle type	Categorical/text; standardize whitespace and labels such as “Unspecified”.

DATA LOAD

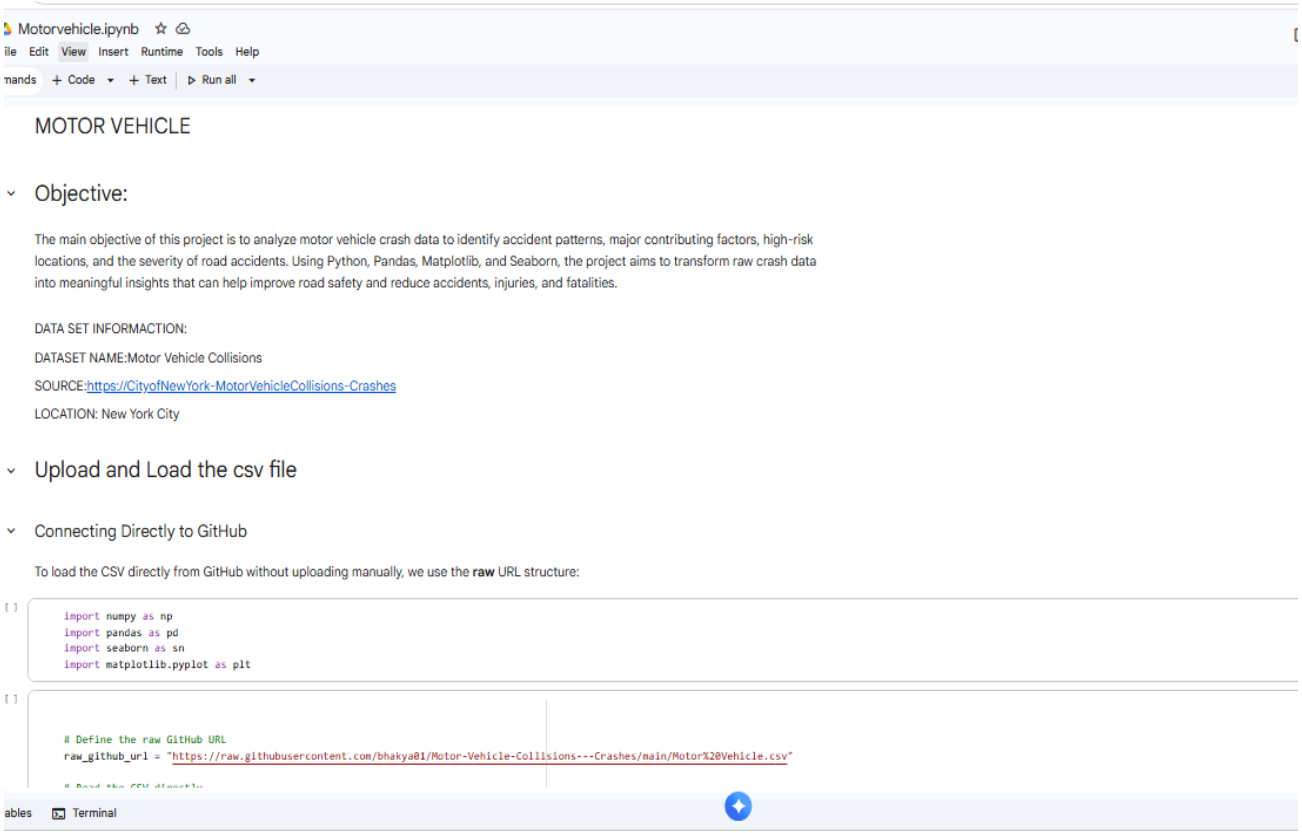
Connecting Directly to GitHub

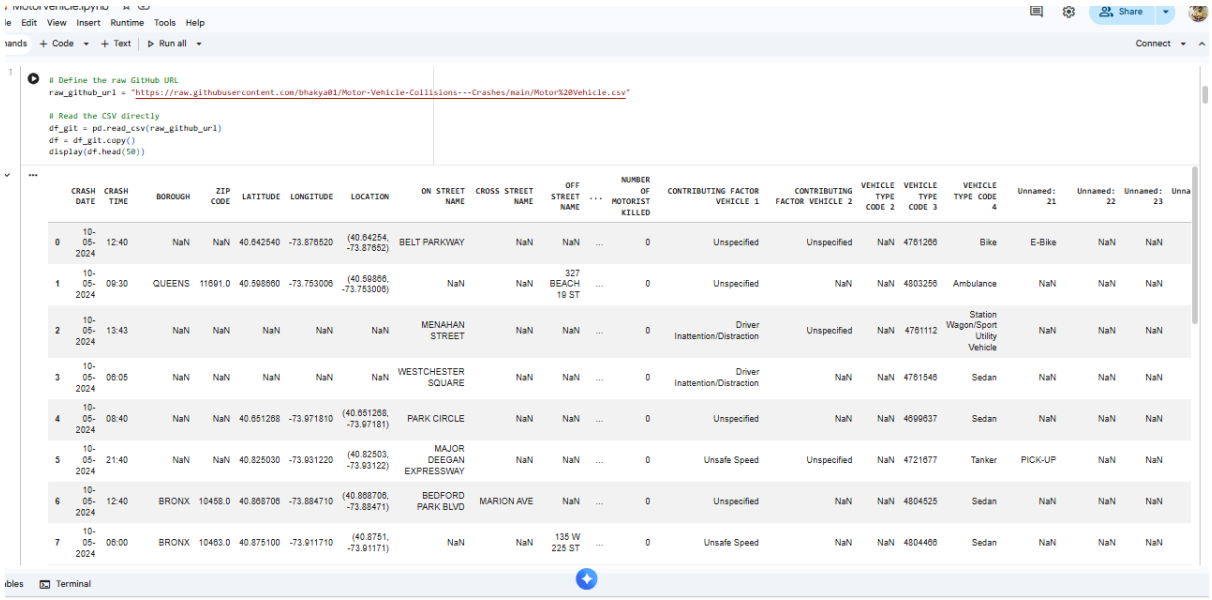
To load the CSV directly from GitHub without uploading manually, we use the raw URL structure:

DATASET NAME: NYC ROAD SAFETY INTELLIGENCE

SOURCE : https://CityofNewYork-MotorVehicleCollisions-Crashes

LOCATION: New York City





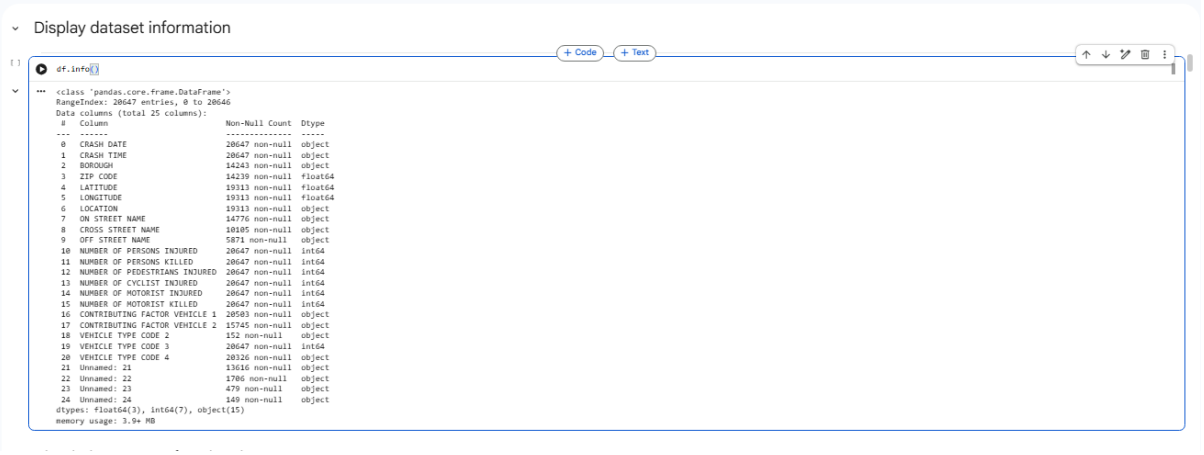
```
# Define the raw GitHub URL
raw_github_url = "https://raw.githubusercontent.com/bhakya01/Motor-Vehicle-Collisions---Crashes/main/Motor%20Vehicle.csv"

# Read the CSV directly
df_git = pd.read_csv(raw_github_url)
df = df_git.copy()
display(df.head(50))
```

	CRASH DATE	CRASH TIME	BOROUGH	ZIP CODE	LATITUDE	LONGITUDE	LOCATION	ON STREET NAME	CROSS STREET NAME	OFF STREET NAME	...	NUMBER OF MOTORIST KILLED	CONTRIBUTING FACTOR VEHICLE 1	CONTRIBUTING FACTOR VEHICLE 2	VEHICLE TYPE CODE 2	VEHICLE TYPE CODE 3	VEHICLE TYPE CODE 4	Unnamed: 21	Unnamed: 22	Unnamed: 23	Unna
0	10-05-2024	12:40	NaN	NaN	40.642540	-73.876520	(40.64254, -73.87652)	BELT PARKWAY	NaN	NaN	...	0	Unspecified	Unspecified	NaN	4761266	Bike	E-Bike	NaN	NaN	
1	10-05-2024	09:30	QUEENS	11691.0	40.598860	-73.753006	(40.59886, -73.753006)	NaN	NaN	327 BEACH 19 ST	...	0	Unspecified	NaN	NaN	4803256	Ambulance	NaN	NaN	NaN	
2	10-05-2024	13:43	NaN	NaN	NaN	NaN	NaN	MENAHAN STREET	NaN	NaN	...	0	Driver Inattention/Distraction	Unspecified	NaN	4761112	Station Wagon/Sport Utility Vehicle	NaN	NaN	NaN	
3	10-05-2024	06:05	NaN	NaN	NaN	NaN	NaN	WESTCHESTER SQUARE	NaN	NaN	...	0	Driver Inattention/Distraction	NaN	NaN	4761546	Sedan	NaN	NaN	NaN	
4	10-05-2024	08:40	NaN	NaN	40.651268	-73.971810	(40.651268, -73.97181)	PARK CIRCLE	NaN	NaN	...	0	Unspecified	NaN	NaN	4699637	Sedan	NaN	NaN	NaN	
5	10-05-2024	21:40	NaN	NaN	40.825030	-73.931220	(40.82503, -73.93122)	MAJOR DEEGAN EXPRESSWAY	NaN	NaN	...	0	Unsafe Speed	Unspecified	NaN	4721677	Tanker	PICK-UP	NaN	NaN	
6	10-05-2024	12:40	BRONX	10458.0	40.868706	-73.884710	(40.868706, -73.88471)	BEDFORD PARK BLVD	MARION AVE	NaN	...	0	Unspecified	NaN	NaN	4804525	Sedan	NaN	NaN	NaN	
7	10-05-2024	06:00	BRONX	10463.0	40.875100	-73.911710	(40.8751, -73.91171)	NaN	NaN	135 W 225 ST	...	0	Unsafe Speed	NaN	NaN	4804466	Sedan	NaN	NaN	NaN	

ibles Terminal

DATA SET INFORMATION

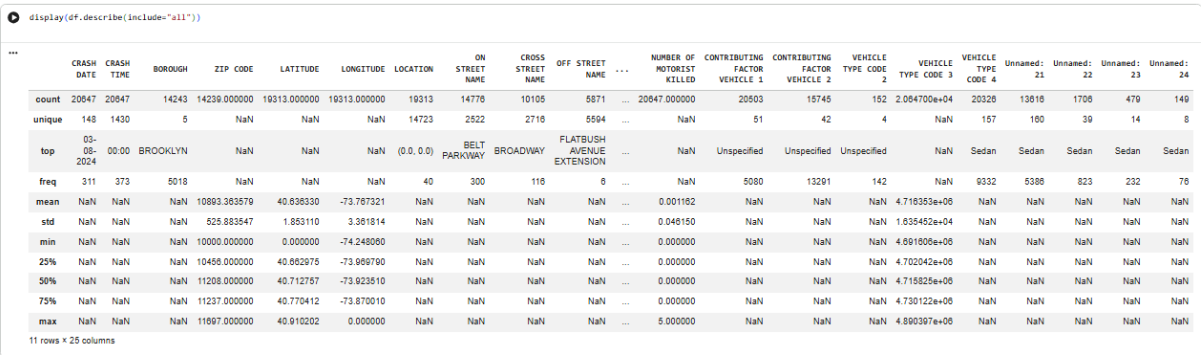


Display dataset information

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20647 entries, 0 to 20646
Data columns (total 25 columns):
 #   Column                         Non-Null Count  Dtype
---  ------                         --------------  -----
 0   CRASH DATE                     20647 non-null  object
 1   CRASH TIME                     20647 non-null  object
 2   BOROUGH                        14243 non-null  object
 3   ZIP CODE                       14239 non-null  float64
 4   LATITUDE                       19313 non-null  float64
 5   LONGITUDE                      19313 non-null  float64
 6   LOCATION                       19313 non-null  object
 7   ON STREET NAME                 14776 non-null  object
 8   CROSS STREET NAME              10105 non-null  object
 9   OFF STREET NAME                5871 non-null   object
 10  NUMBER OF PERSONS INJURED      20647 non-null  int64
 11  NUMBER OF PERSONS KILLED       20647 non-null  int64
 12  NUMBER OF PEDESTRIANS INJURED  20647 non-null  int64
 13  NUMBER OF CYCLIST INJURED      20647 non-null  int64
 14  NUMBER OF MOTORIST INJURED     20647 non-null  int64
 15  NUMBER OF MOTORIST KILLED      20647 non-null  int64
 16  CONTRIBUTING FACTOR VEHICLE 1  20503 non-null  object
 17  CONTRIBUTING FACTOR VEHICLE 2  15745 non-null  object
 18  VEHICLE TYPE CODE 2            152 non-null    object
 19  VEHICLE TYPE CODE 3            20647 non-null  int64
 20  VEHICLE TYPE CODE 4            20326 non-null  object
 21  Unnamed: 21                    13616 non-null  object
 22  Unnamed: 22                    1706 non-null   object
 23  Unnamed: 23                    479 non-null    object
 24  Unnamed: 24                    149 non-null    object
dtypes: float64(3), int64(7), object(15)
memory usage: 3.9+ MB
```

Generate descriptive statistics

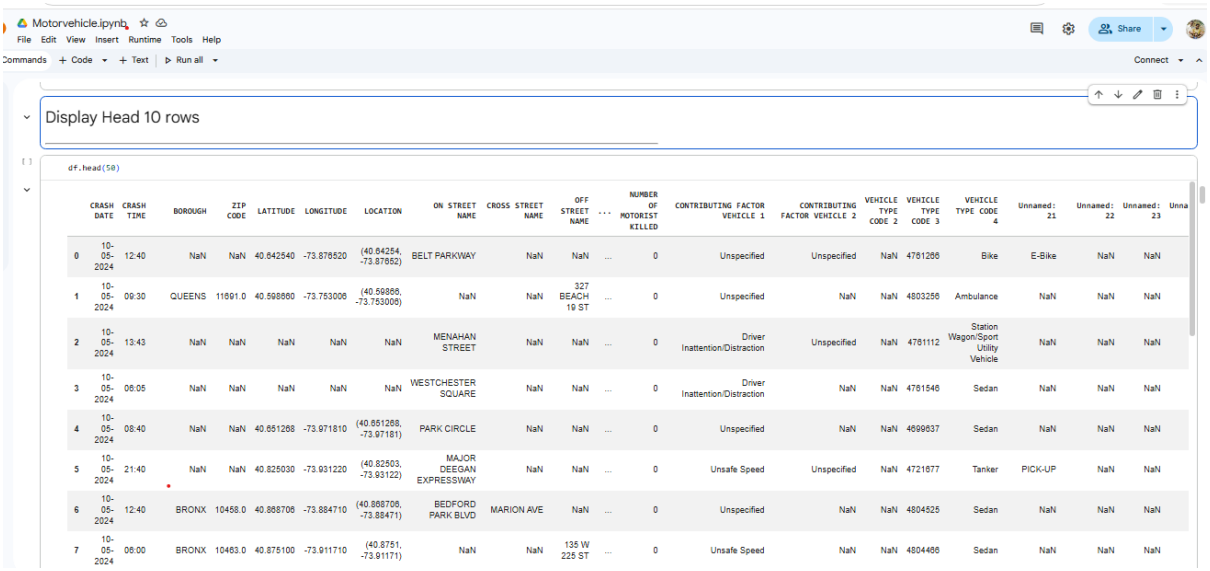


```
display(df.describe(include="all"))
```

	CRASH DATE	CRASH TIME	BOROUGH	ZIP CODE	LATITUDE	LONGITUDE	LOCATION	ON STREET NAME	CROSS STREET NAME	OFF STREET NAME	...	NUMBER OF MOTORIST KILLED	CONTRIBUTING FACTOR VEHICLE 1	CONTRIBUTING FACTOR VEHICLE 2	VEHICLE TYPE CODE 2	VEHICLE TYPE CODE 3	VEHICLE TYPE CODE 4	Unnamed: 21	Unnamed: 22	Unnamed: 23	Unnamed: 24
count	20647	20647	14243	14239.000000	19313.000000	19313.000000	19313	14776	10105	5871	...	20647.000000	20503	15745	152	2.064700e+04	20326	13616	1706	479	149
unique	148	1430	5	NaN	NaN	NaN	14723	2522	2716	5594	...	NaN	51	42	4	NaN	157	160	39	14	8
top	03-08-2024	00:00	BROOKLYN	NaN	NaN	NaN	(0.0, 0.0)	BELT PARKWAY	BROADWAY	FLATBUSH AVENUE EXTENSION	...	NaN	Unspecified	Unspecified	Unspecified	NaN	Sedan	Sedan	Sedan	Sedan	Sedan
freq	311	373	5018	NaN	NaN	NaN	40	300	116	8	...	NaN	5080	13291	142	NaN	9332	5386	823	232	76
mean	NaN	NaN	NaN	10893.363579	40.636330	-73.767321	NaN	NaN	NaN	NaN	...	0.001162	NaN	NaN	NaN	4.716353e+06	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	525.883547	1.853110	3.361814	NaN	NaN	NaN	NaN	...	0.046150	NaN	NaN	NaN	1.635452e+04	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	10000.000000	0.000000	-74.248060	NaN	NaN	NaN	NaN	...	0.000000	NaN	NaN	NaN	4.691606e+06	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	10456.000000	40.662975	-73.969790	NaN	NaN	NaN	NaN	...	0.000000	NaN	NaN	NaN	4.702042e+06	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	11208.000000	40.712757	-73.923510	NaN	NaN	NaN	NaN	...	0.000000	NaN	NaN	NaN	4.715825e+06	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	11237.000000	40.770412	-73.870010	NaN	NaN	NaN	NaN	...	0.000000	NaN	NaN	NaN	4.730122e+06	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	11697.000000	40.910202	0.000000	NaN	NaN	NaN	NaN	...	5.000000	NaN	NaN	NaN	4.890397e+06	NaN	NaN	NaN	NaN	NaN

11 rows × 25 columns

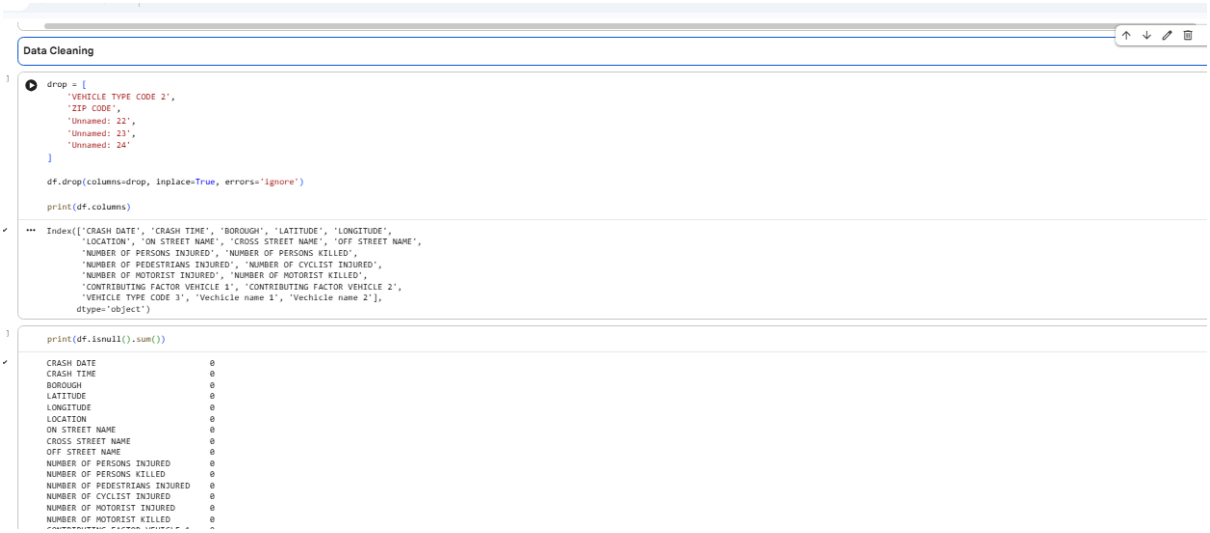
DISPLAY DATA SET FIRST 10 ROW



	CRASH DATE	CRASH TIME	BOROUGH	ZIP CODE	LATITUDE	LONGITUDE	LOCATION	ON STREET NAME	CROSS STREET NAME	OFF STREET NAME	...	NUMBER OF MOTORIST KILLED	CONTRIBUTING FACTOR VEHICLE 1	CONTRIBUTING FACTOR VEHICLE 2	VEHICLE TYPE CODE 2	VEHICLE TYPE CODE 3	VEHICLE TYPE CODE 4	Unnamed: 21	Unnamed: 22	Unnamed: 23	Unna
0	10-05-2024	12:40	NaN	NaN	40.642540	-73.876520	(40.64254, -73.87652)	BELT PARKWAY	NaN	NaN	...	0	Unspecified	Unspecified	NaN	4761266	Bike	E-Bike	NaN	NaN	
1	10-05-2024	09:30	QUEENS	11691.0	40.598660	-73.753006	(40.59866, -73.753006)	NaN	NaN	327 BEACH 19 ST	...	0	Unspecified	NaN	NaN	4803256	Ambulance	NaN	NaN	NaN	
2	10-05-2024	13:43	NaN	NaN	NaN	NaN	NaN	MENAHAN STREET	NaN	NaN	...	0	Driver Inattention/Distraction	Unspecified	NaN	4781112	Station Wagon/Sport Utility Vehicle	NaN	NaN	NaN	
3	10-05-2024	06:05	NaN	NaN	NaN	NaN	NaN	WESTCHESTER SQUARE	NaN	NaN	...	0	Driver Inattention/Distraction	NaN	NaN	4781546	Sedan	NaN	NaN	NaN	
4	10-05-2024	08:40	NaN	NaN	40.651268	-73.971810	(40.651268, -73.97181)	PARK CIRCLE	NaN	NaN	...	0	Unspecified	NaN	NaN	4699637	Sedan	NaN	NaN	NaN	
5	10-05-2024	21:40	NaN	NaN	40.825030	-73.931220	(40.82503, -73.93122)	MAJOR DEEGAN EXPRESSWAY	NaN	NaN	...	0	Unsafe Speed	Unspecified	NaN	4721677	Tanker	PICK-UP	NaN	NaN	
6	10-05-2024	12:40	BRONX	10458.0	40.868706	-73.884710	(40.868706, -73.88471)	BEDFORD PARK BLVD	MARION AVE	NaN	...	0	Unspecified	NaN	NaN	4804525	Sedan	NaN	NaN	NaN	
7	10-05-2024	06:00	BRONX	10463.0	40.875100	-73.911710	(40.8751, -73.91171)	NaN	NaN	135 W 225 ST	...	0	Unsafe Speed	NaN	NaN	4804466	Sedan	NaN	NaN	NaN	

DATA CLEANNING

- Data set cleaning **:** In my NYC ROAD SAFETY INTELLIGENCE I drop my (vehicle type code 2,zip code , unnamed 22,unnamed 23,unnamed 24 has been ignore
- I cleaned using is null
- Missing value after cleaning and Total missing values



```
Vechicle name 1                0
Vechicle name 2                0
dtype: int64
```

```
cols =['BOROUGH',
       'LATITUDE','LONGITUDE','LOCATION','ON STREET NAME' ,
'CROSS STREET NAME','OFF STREET NAME','CONTRIBUTING FACTOR VEHICLE 1'
,'CONTRIBUTING FACTOR VEHICLE 2','VEHICLE TYPE CODE 3','Vechicle name 1','Vechicle name 2']
df[cols]=df[cols].fillna('unknown')
```

```
print(" Missing values after cleaning")
print(df.isnull().sum())
print("\nTotal  missing  values")
print(df.isnull().sum())
```

```
 Missing values after cleaning
CRASH DATE                     0
CRASH TIME                     0
BOROUGH                        0
LATITUDE                       0
LONGITUDE                      0
LOCATION                       0
ON STREET NAME                 0
CROSS STREET NAME              0
OFF STREET NAME                0
NUMBER OF PERSONS INJURED      0
NUMBER OF PERSONS KILLED       0
NUMBER OF PEDESTRIANS INJURED  0
NUMBER OF CYCLIST INJURED      0
NUMBER OF MOTORIST INJURED     0
NUMBER OF MOTORIST KILLED      0
CONTRIBUTING FACTOR VEHICLE 1  0
CONTRIBUTING FACTOR VEHICLE 2  0
VEHICLE TYPE CODE 3            0
Vechicle name 1                0
Vechicle name 2                0
dtype: int64

Total  missing  values
CRASH DATE                     0
CRASH TIME                     0
BOROUGH                        0
```

FEATURE ENGINEERING(EDA)

RENAME : In NYC ROAD SAFETY INTELLIGENCE Data set record I rename columns (Unamed23 has Vechicle name 2 and VECHICLETYPE CODE 4 has Vechicle name1)

Motorvehicle.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

Share Connect

Feature Engineeing (EDA)

```
df.rename(columns={
    'Unnamed: 21': 'Vechicle name 2',
    'VEHICLE TYPE CODE 4': 'Vechicle name 1'
}, inplace=True)

print(df.columns)
```

```
Index(['CRASH DATE', 'CRASH TIME', 'BOROUGH', 'LATITUDE', 'LONGITUDE',
       'LOCATION', 'ON STREET NAME', 'CROSS STREET NAME', 'OFF STREET NAME',
       'NUMBER OF PERSONS INJURED', 'NUMBER OF PERSONS KILLED',
       'NUMBER OF PEDESTRIANS INJURED', 'NUMBER OF CYCLIST INJURED',
       'NUMBER OF MOTORIST INJURED', 'NUMBER OF MOTORIST KILLED',
       'CONTRIBUTING FACTOR VEHICLE 1', 'CONTRIBUTING FACTOR VEHICLE 2',
       'VEHICLE TYPE CODE 3', 'Vechicle name 1', 'Vechicle name 2'],
      dtype='object')
```

Afternoon Commute Vulnerability: The Afternoon period (12:00 PM to 4:59 PM) has the highest concentration of crashes with 5,809 incidents, closely followed by the Morning period (5:00 AM to 11:59 AM) with 5,718 incidents. Together, these daytime/commuting periods account for over 55% of all crashes.

Lower Risk at Twilight: The Evening window (5:00 PM to 8:59 PM) shows the lowest risk with 4,296 crashes, indicating that while traffic density is high, the shorter duration of this transition block lowers absolute crash counts.

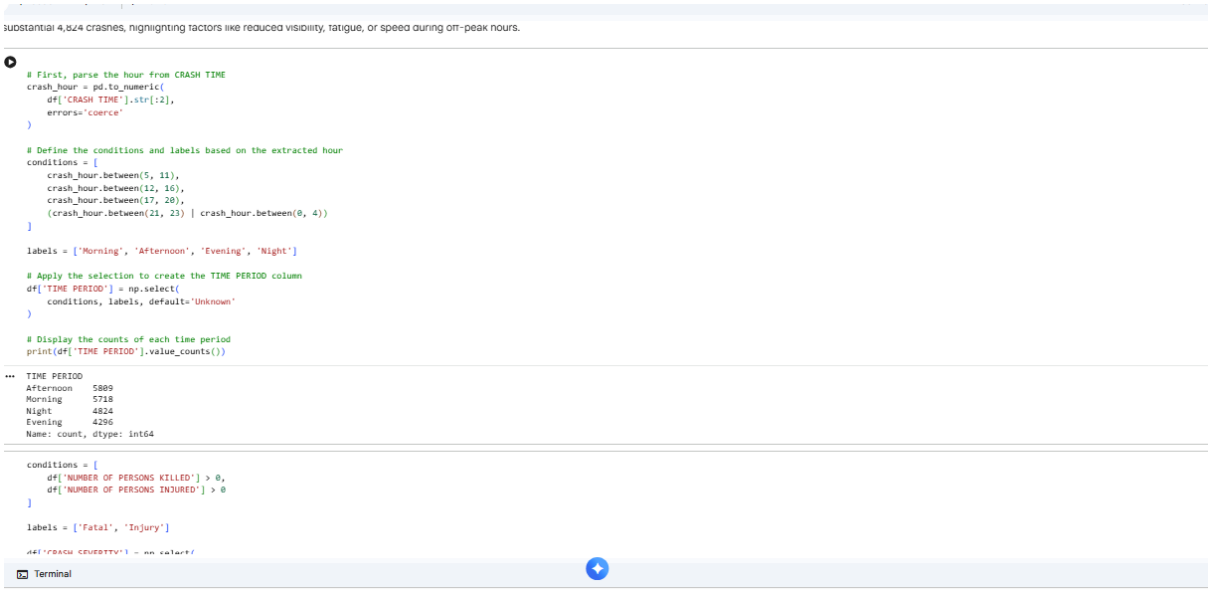
Nighttime Disproportion: Despite significantly lower road occupancy overnight, the Night period (9:00 PM to 4:59 AM) still records a substantial 4,824 crashes, highlighting factors like reduced visibility, fatigue, or speed during off-peak hours.

```
# First, parse the hour from CRASH TIME
crash_hour = pd.to_numeric(
    df['CRASH TIME'].str[:2],
    errors='coerce'
)

# Define the conditions and labels based on the extracted hour
conditions = [
    crash_hour.between(5, 11),
```

Crash hour :crash time has been splitted in 4 parts.

- Crash hour between(5,11)has Morning,
- Crash hour between(12,16)has Afternoon,
- Crash hour between(17,20)has Evenning,
- Crash hour between(21,24)has Night



substantial 4,824 crashes, highlighting factors like reduced visibility, fatigue, or speed during off-peak hours.

```
# First, parse the hour from CRASH TIME
crash_hour = pd.to_numeric(
    df['CRASH TIME'].str[:2],
    errors='coerce'
)

# Define the conditions and labels based on the extracted hour
conditions = [
    crash_hour.between(5, 11),
    crash_hour.between(12, 16),
    crash_hour.between(17, 20),
    (crash_hour.between(21, 23) | crash_hour.between(0, 4))
]

labels = ['Morning', 'Afternoon', 'Evening', 'Night']

# Apply the selection to create the TIME PERIOD column
df['TIME PERIOD'] = np.select(
    conditions, labels, default='Unknown'
)

# Display the counts of each time period
print(df['TIME PERIOD'].value_counts())
```

```
TIME PERIOD
Afternoon    5809
Morning      5718
Night        4824
Evening      4296
Name: count, dtype: int64
```

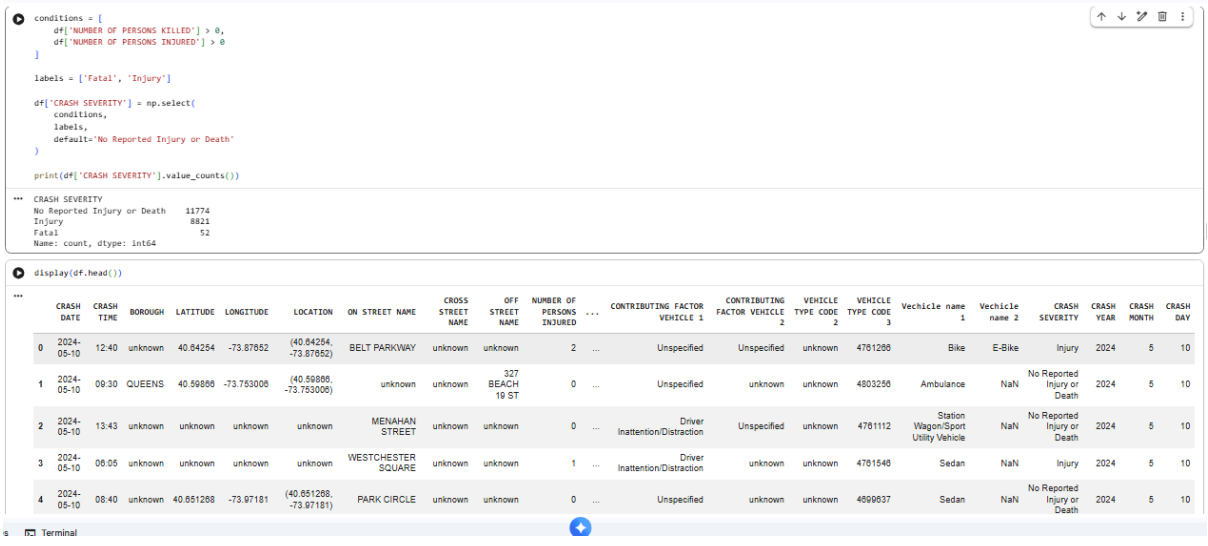
```
conditions = [
    df['NUMBER OF PERSONS KILLED'] > 0,
    df['NUMBER OF PERSONS INJURED'] > 0
]

labels = ['Fatal', 'Injury']
```

Terminal

Crash severity: Number of persons killed and Number of persons injured has been separated into 3 types

- NoReported Injury or death:11774
- Injury:8825
- Fatal:52



```
conditions = [
    df['NUMBER OF PERSONS KILLED'] > 0,
    df['NUMBER OF PERSONS INJURED'] > 0
]

labels = ['Fatal', 'Injury']

df['CRASH SEVERITY'] = np.select(
    conditions,
    labels,
    default='No Reported Injury or Death'
)

print(df['CRASH SEVERITY'].value_counts())
```

```
CRASH SEVERITY
No Reported Injury or Death    11774
Injury                          8821
Fatal                             52
Name: count, dtype: int64
```

```
display(df.head())
```

	CRASH DATE	CRASH TIME	BOROUGH	LATITUDE	LONGITUDE	LOCATION	ON STREET NAME	CROSS STREET NAME	OFF STREET NAME	NUMBER OF PERSONS INJURED	...	CONTRIBUTING FACTOR VEHICLE 1	CONTRIBUTING FACTOR VEHICLE 2	VEHICLE TYPE CODE 2	VEHICLE TYPE CODE 3	Vechicle name 1	Vechicle name 2	CRASH SEVERITY	CRASH YEAR	CRASH MONTH	CRASH DAY
0	2024-05-10	12:40	unknown	40.64254	-73.87652	(40.64254, -73.87652)	BELT PARKWAY	unknown	unknown	2	...	Unspecified	Unspecified	unknown	4761266	Bike	E-Bike	Injury	2024	5	10
1	2024-05-10	09:30	QUEENS	40.59866	-73.753006	(40.59866, -73.753006)	unknown	unknown	327 BEACH 19 ST	0	...	Unspecified	unknown	unknown	4803256	Ambulance	NaN	No Reported Injury or Death	2024	5	10
2	2024-05-10	13:43	unknown	unknown	unknown	unknown	MENAHAN STREET	unknown	unknown	0	...	Driver Inattention/Distraction	Unspecified	unknown	4761112	Station Wagon/Sport Utility Vehicle	NaN	No Reported Injury or Death	2024	5	10
3	2024-05-10	06:05	unknown	unknown	unknown	unknown	WESTCHESTER SQUARE	unknown	unknown	1	...	Driver Inattention/Distraction	unknown	unknown	4761546	Sedan	NaN	Injury	2024	5	10
4	2024-05-10	08:40	unknown	40.651268	-73.97181	(40.651268, -73.97181)	PARK CIRCLE	unknown	unknown	0	...	Unspecified	unknown	unknown	4699637	Sedan	NaN	No Reported Injury or Death	2024	5	10

Terminal

Crash Date has been extraction

- Crash year (2024-2026)
- Crash month (1-12)
- Crash date (1-30)

YEAR,MONTH DATE EXTERACTION

```
# Convert CRASH DATE to datetime format (DD-MM-YYYY)
df['CRASH DATE'] = pd.to_datetime(df['CRASH DATE'], format='%d-%m-%Y', errors='coerce')

# Extract year and month
df['CRASH YEAR'] = df['CRASH DATE'].dt.year
df['CRASH MONTH'] = df['CRASH DATE'].dt.month
df['CRASH DAY'] = df['CRASH DATE'].dt.day

# Display the extracted columns alongside the original date
display(df[['CRASH DATE', 'CRASH YEAR', 'CRASH MONTH', 'CRASH DAY']].head(10))
```

	CRASH DATE	CRASH YEAR	CRASH MONTH	CRASH DAY
0	2024-05-10	2024	5	10
1	2024-05-10	2024	5	10
2	2024-05-10	2024	5	10
3	2024-05-10	2024	5	10
4	2024-05-10	2024	5	10
5	2024-05-10	2024	5	10
6	2024-05-10	2024	5	10
7	2024-05-10	2024	5	10
8	2024-05-10	2024	5	10
9	2024-05-10	2024	5	10

```
print(['Year',
       'Month',
       'Day',
       'Time period',
       'CRASH TIME',
       'CRASH SEVERITY',
       'TOTAL CASUALTIES'])

display(df.head())
```

```
['Year', 'Month', 'Day', 'Time period', 'CRASH TIME', 'CRASH SEVERITY', 'TOTAL CASUALTIES']
```

	CRASH DATE	CRASH TIME	BOROUGH	LATITUDE	LONGITUDE	LOCATION	ON STREET NAME	CROSS STREET NAME	OFF STREET NAME	NUMBER OF PERSONS INJURED	...	NUMBER OF PEDESTRIANS INJURED	NUMBER OF CYCLIST INJURED	NUMBER OF MOTORIST INJURED	NUMBER OF MOTORIST KILLED	CONTRIBUTING FACTOR VEHICLE 1	CONTRIBUTING FACTOR VEHICLE 2	VEHICLE TYPE CODE 3	Vechicle name 1	Vechicle name 2	TIME PERIOD
0	10-05-2024	12:40	unknown	40.64254	-73.87652	(40.64254, -73.87652)	BELT PARKWAY	unknown	unknown	2	...	0	1	0	0	Unspecified	Unspecified	4761286	Bike	E-Bike	Afternoon
1	10-05-2024	09:30	QUEENS	40.59866	-73.753006	(40.59866, -73.753006)	unknown	unknown	327 BEACH 19 ST	0	...	0	0	0	0	Unspecified	unknown	4803256	Ambulance	unknown	Morning
2	10-05-2024	13:43	unknown	unknown	unknown	unknown	MENAHAN STREET	unknown	unknown	0	...	0	0	0	0	Driver Inattention/Distraction	Unspecified	4761112	Station Wagon/Sport Utility Vehicle	unknown	Afternoon
3	10-05-2024	08:05	unknown	unknown	unknown	unknown	WESTCHESTER SQUARE	unknown	unknown	1	...	1	0	0	0	Driver Inattention/Distraction	unknown	4761546	Sedan	unknown	Morning
4	10-05-2024	08:40	unknown	40.651288	-73.97181	(40.651288, -73.97181)	PARK CIRCLE	unknown	unknown	0	...	0	0	0	0	Unspecified	unknown	4699637	Sedan	unknown	Morning

5 rows × 21 columns

Data visualizations:

Number of crash year and crash count in year of 2024 the crash count 20000 has been increased and In the year of 2025 in 350 has been less them when compared to last one year and there is no crash count in 2026

DATA VISUALIZATIONS

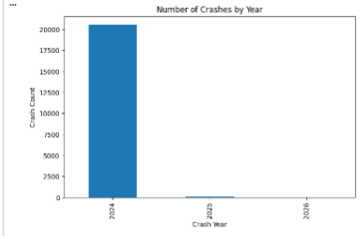
Extreme Concentration in 2024: The vast majority of vehicle crashes in this dataset occurred in 2024 (comprising over 99% of the records).

Data Completion Dropoff: The years 2025 and 2026 show exceptionally low crash counts (dropping down to single digits). Rather than a miraculous reduction in accidents, this points to a data limitation where the dataset only covers a tiny, incomplete portion of early 2025/2026.

Planning Implications: For accurate predictive modeling or trend analysis, you should filter your analyses specifically to 2024 to avoid seasonal skewing caused by the incomplete tracking in subsequent years

```
df['CRASH YEAR'].value_counts().sort_index().plot(kind='bar', figsize=(8,5))

plt.title('Number of Crashes by Year')
plt.xlabel('Crash Year')
plt.ylabel('Crash Count')
plt.show()
```



Monthly wise crash distribution

- In this monthy wise crash distibution in(May to June)in spring and summer month
- (October to November) is holiday is increased trips
- Lower Accident count(Feb and Jan) winter weather



Bar chart crash time period and crash count

- Accidents has been peak in Afteroon(5,809)and Morning(5,718) Because its heavy traffic
- Lower Accidents in Evenning(4,290)and Night(4,824) lower collsion



Number of vehicle crash and vehicle type

- Sedan is above 8000,station wagons utility vehicle less than 7000
- Tax,pick up truck and bus(1500)
- Bike, box truck, motorcycle(500)
- Moped, Tractor ,Truck Diesel(100)

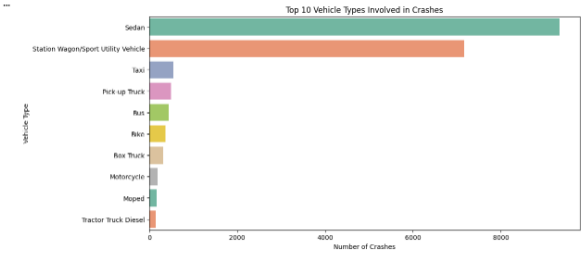
```
plt.figure(figsize=(12,6))

vehicle = df['Vechicle name 1'].replace(
    ['unknown', 'Unspecified'], pd.NA
).dropna().value_counts().head(10)

sns.barplot(
    x=vehicle.values,
    y=vehicle.index,
    hue=vehicle.index,
    palette='Set2',
    legend=False
)

plt.title('Top 10 Vehicle Types Involved in Crashes')
plt.xlabel('Number of Crashes')
plt.ylabel('Vehicle Type')

plt.show()
```



Pivot table

- Most Dangerous Collision Dynamics in Sport Utility Vehicles (1,211 injuries) and sedans(1794 injuries)
- . Because of mutual collisions dominate the overall safety metrics, producing the largest clusters of motorist injuries in the entire dataset.

pivot table:

Most Dangerous Collision Dynamics: The highest volume of motorist injuries occurs in collisions involving** Station Wagon/Sport Utility Vehicles **colliding with other Station Wagon/Sport Utility Vehicles (1,211 injuries) or with Sedans**

The Common Factor: Because Sedans and Station Wagons/SUVs make up the their mutual collisions dominate the overall safety metrics, producing the largest clusters of motorist injuries in the entire dataset.

```
# Remove extra spaces from column names
df.columns = df.columns.str.strip()

# Convert injury column to numeric
df['NUMBER OF MOTORIST INJURED'] = pd.to_numeric(
    df['NUMBER OF MOTORIST INJURED'],
    errors='coerce'
)

# Get top 10 vehicle types
top1 = df['Vechicle name 1'].value_counts().head(10).index
top2 = df['Vechicle name 2'].value_counts().head(10).index

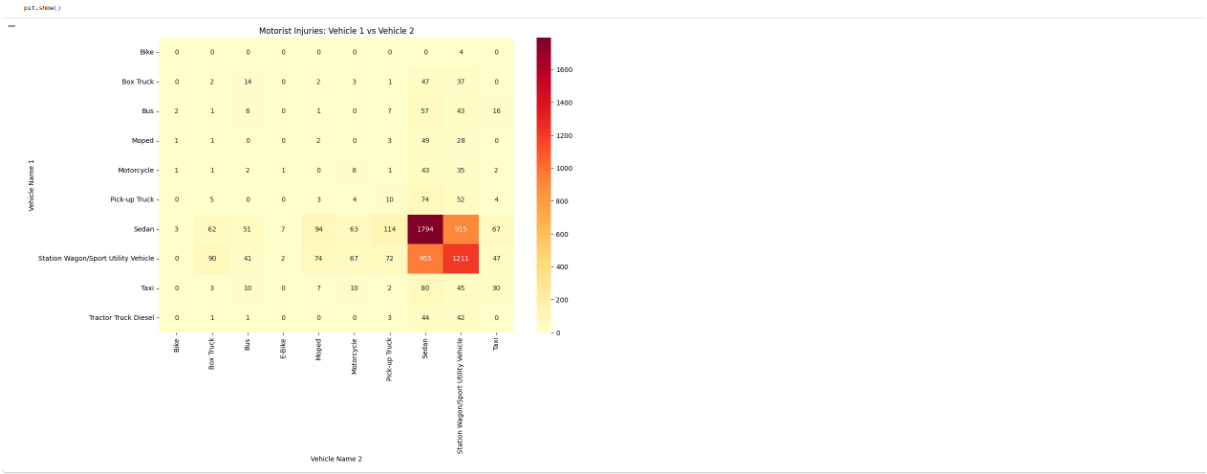
# Filter data
filtered = df[
    df['Vechicle name 1'].isin(top1) &
    df['Vechicle name 2'].isin(top2)
]

# Create pivot table
injury_pivot = pd.pivot_table(
    filtered,
    index='Vechicle name 1',
    columns='Vechicle name 2',
    values='NUMBER OF MOTORIST INJURED',
    aggfunc='sum',
    fill_value=0
)

# Heatmap
plt.figure(figsize=(12,8))

sns.heatmap(
    injury_pivot,
    annot=True,
    fmt='.0f',
    cmap="YlOrRd"
)

plt.title('Motorist Injuries: Vehicle 1 vs Vehicle 2')
plt.xlabel('Vehicle Name 2')
plt.ylabel('Vehicle Name 1')
plt.show()
```



GROUP BY

Vechicle name 1,Vechicle name 2 has been grouped by Number of Motorist killed

```
df['NUMBER OF MOTORIST KILLED'] = pd.to_numeric(
    df['NUMBER OF MOTORIST KILLED'],
    errors='coerce'
)

killed_group = df.groupby(
    ['Vechicle name 1', 'Vechicle name 2']
)['NUMBER OF MOTORIST KILLED'].sum().reset_index()

killed_group = killed_group.sort_values(
    by='NUMBER OF MOTORIST KILLED',
    ascending=False
)

print(killed_group.head(10))
```

```
                    Vechicle name 1                    Vechicle name 2  \
395                           Sedan  Station Wagon/Sport Utility Vehicle
557                 Tractor Truck Diesel                      Motorcycle
263                      Motorscooter                             Bus
261                        Motorcycle                 Tractor Truck Diesel
392                           Sedan                             Sedan
485  Station Wagon/Sport Utility Vehicle                             Sedan
363                           Sedan                 Garbage or Refuse
489  Station Wagon/Sport Utility Vehicle  Station Wagon/Sport Utility Vehicle
406                           Sedan                         UHAUL VAN
407                           Sedan                               UNK

     NUMBER OF MOTORIST KILLED
395                          5
557                          1
263                          1
261                          1
392                          1
485                          1
363                          1
489                          1
406                          0
407                          0
```

Bar chat

- Highest Fatalities: Sedan , Station Wagon/Sport Utility Vehicle recorded the highest number, with 5 motorist deaths
- Other Vehicle Tractor Truck Diesel, Motorcycle recorded 1 death.
  Motorcycle-related Fatalities: Motor scooter , Bus and Motorcycle , Tractor Truck Diesel each recorded 1 death
- Low Fatality Counts: Most of the displayed vehicle combinations recorded only 1 death, while some recorded zero.

Draw back in Sedan

Possible Risk Factors: Differences in vehicle size, weight, visibility, collision speed and impact direction may influence crasseverity

BAR CHART

1. Highest Fatalities: Sedan + Station Wagon/Sport Utility Vehicle recorded the highest number, with 5 motorist deaths.

2. Other Vehicle Combinations: Tractor Truck Diesel + Motorcycle recorded 1 death.

3. Motorcycle-related Fatalities: Motorscooter + Bus and Motorcycle + Tractor Truck Diesel each recorded 1 death.

4. Low Fatality Counts: Most of the displayed vehicle combinations recorded only 1 death, while some recorded zero.

Draw back in Sedan Possible Risk Factors:Differences in vehicle size, weight, visibility, collision speed and impact direction may influence crash severity.

Recommendation: Improve: driver awareness, maintain safe following distances, follow speed limits and promote the use of seatbelts.

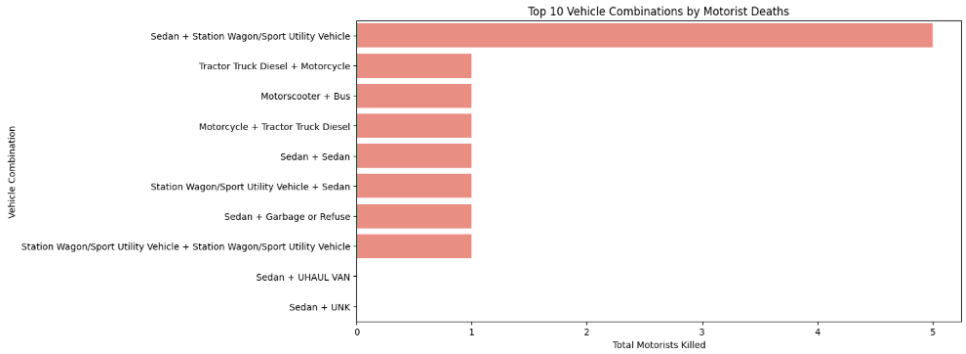
```
top10 = killed_group.head(10).copy()

top10['Vehicle Combination'] = (
    top10['Vechicle name 1'].astype(str) + ' + ' +
    top10['Vechicle name 2'].astype(str)
)

plt.figure(figsize=(12,6))

sns.barplot(
    data=top10,
    x='NUMBER OF MOTORIST KILLED',
    y='Vehicle Combination',
    color='salmon'
)

plt.title('Top 10 Vehicle Combinations by Motorist Deaths')
plt.xlabel('Total Motorists Killed')
plt.ylabel('Vehicle Combination')
plt.show()
```



Line chart

- Highest Death Count: January 2024 recorded the highest daily motorist death count, with 5 deaths on a single date.
- Low Daily Fatalities Most recorded dates show 0 or 1 motorist death.
- 2025–2026 TrendThe graph shows mostly zero deaths on the recorded dates during 2025 and early 2026

Line chart

1. Highest Death Count: January 2024 recorded the highest daily motorist death count, with 5 deaths on a single date.

2. Low Daily Fatalities: Most recorded dates show 0 or 1 motorist death.

3. 2025–2026 Trend: The graph shows mostly zero deaths on the recorded dates during 2025 and early 2026.

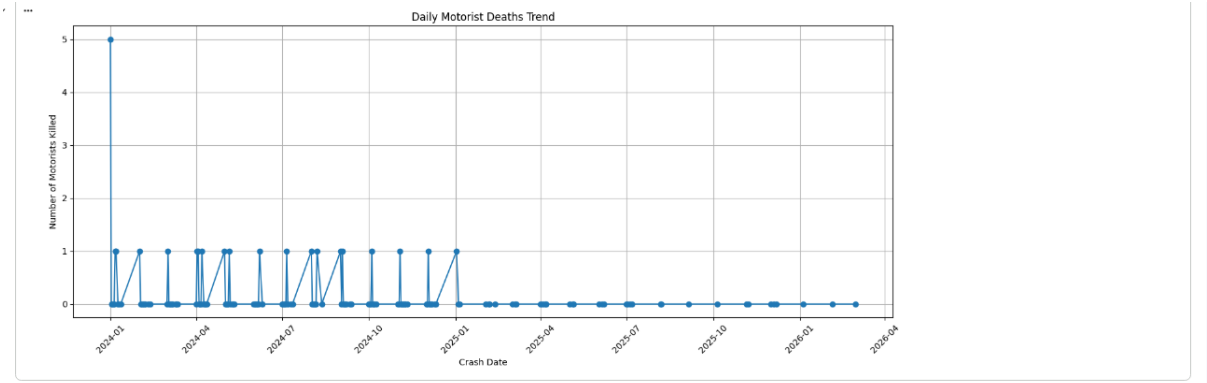
```
df['CRASH DATE'] = pd.to_datetime(
    df['CRASH DATE'], errors='coerce', dayfirst=True
)

daily_deaths = df.groupby('CRASH DATE')[
    'NUMBER OF MOTORIST KILLED'
].sum()

plt.figure(figsize=(14,6))

plt.plot(
    daily_deaths.index,
    daily_deaths.values,
    marker='o'
)

plt.title('Daily Motorist Deaths Trend')
plt.xlabel('Crash Date')
plt.ylabel('Number of Motorists Killed')
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```



Conclusion & Recommendations:

- This analysis reveals that traffic accidents in New York City are heavily dictated by peak commuting times, seasonality, and vehicle size disparities.
- Targeted Patrols: Deploying higher law enforcement presence and speed cameras during peak afternoon
- Seasonal Safety Campaigns: Launching public awareness campaigns ahead of the peak accident months in May/June and October/November.
- Strict Safety Standards for SUVs & Sedans: Promoting defensive driving, maintaining safe following distances, and strictly enforcing seatbelt usage to reduce the severity of impacts between larger SUVs and car size is smaller in sedans.
- The analysis highlights the importance of understanding crash frequency across different time periods, identifying common contributing factors
- That support better road safety awareness, informed decision-making, and effective traffic management strategies.

4.Conclusion

The sample provides useful fields for examining crash timing, location, casualties, and possible contributing factors. Before drawing conclusions, load the original CSV and verify its actual shape, column alignment, data types, missing values, duplicates, and date format. Do not treat the five sample rows as representative of the entire dataset. Important: This report is based only on the five sample rows and column names supplied in the prompt. No full-dataset totals or statistical conclusions have been inferred.